192425037

EXP26:

```
#include <stdio.h>

typedef unsigned long long u64;

long long gcd(long long a, long long b) { return b == 0 ? a : gcd(b, a % b); }

long long egcd(long long a, long long b, long long *x, long long *y) {
    long long x1, y1, g;
    if (b == 0) { *x = 1; *y = 0; return a; }
    g = egcd(b, a % b, &x1, &y1);
    *x = y1; *y = x1 - (a / b) * y1;
    return g;
}

u64 modpow(u64 b, u64 e, u64 m) {
    u64 r = 1; b %= m;
    while (e) { if (e & 1) r = r * b % m; b = b * b % m; e >>= 1; }
    return r;
}

int main() {
    long long n, e, d, k, r, p = 0, q, phi, x, y, newe, newd;
    int s = 0, a, i;

    printf("Bob's public key  (e n): ");
    scanf("%lld %lld", &e, &n);
    printf("Bob's leaked private key d: ");
    scanf("%lld", &d);
```

```
    /* ed - 1 is a multiple of phi(n). Write ed-1 = 2^s * r and look for a
       non-trivial square root of 1 mod n; it reveals a factor of n. */
    k = e * d - 1; r = k;
    while (r % 2 == 0) { r /= 2; s++; }

    for (a = 2; a < 50 && !p; a++) {
        u64 v = modpow(a, r, n);
        if (v == 1 || v == (u64)n - 1) continue;
        for (i = 0; i < s; i++) {
            u64 w = v * v % n;
            if (w == 1) { p = gcd(v - 1, n); break; }
            if (w == (u64)n - 1) break;
            v = w;
        }
        if (p == 1 || p == n) p = 0;
    }
    q = n / p; phi = (p - 1) * (q - 1);
    printf("\nAttacker factors n using only (e, n, d):\n");
    printf("  p = %lld, q = %lld, phi(n) = %lld\n", p, q, phi);

    newe = 17;
    egcd(newe, phi, &x, &y);
    newd = ((x % phi) + phi) % phi;
    printf("Bob picks a new public key e' = %lld (same n).\n", newe);
    printf("Attacker computes the new private key d' = %lld without any help.\n", newd);
    printf("\nIs it safe? NO. Bob must generate a completely new modulus n.\n");
    return 0;
}
```

OUTPUT:

```
Bob's public key  (e n): 31 3599
Bob's leaked private key d: 3031

Attacker factors n using only (e, n, d):
  p = 59, q = 61, phi(n) = 3480
Bob picks a new public key e' = 17 (same n).
Attacker computes the new private key d' = 1433 without any help.

Is it safe? NO. Bob must generate a completely new modulus n.
```